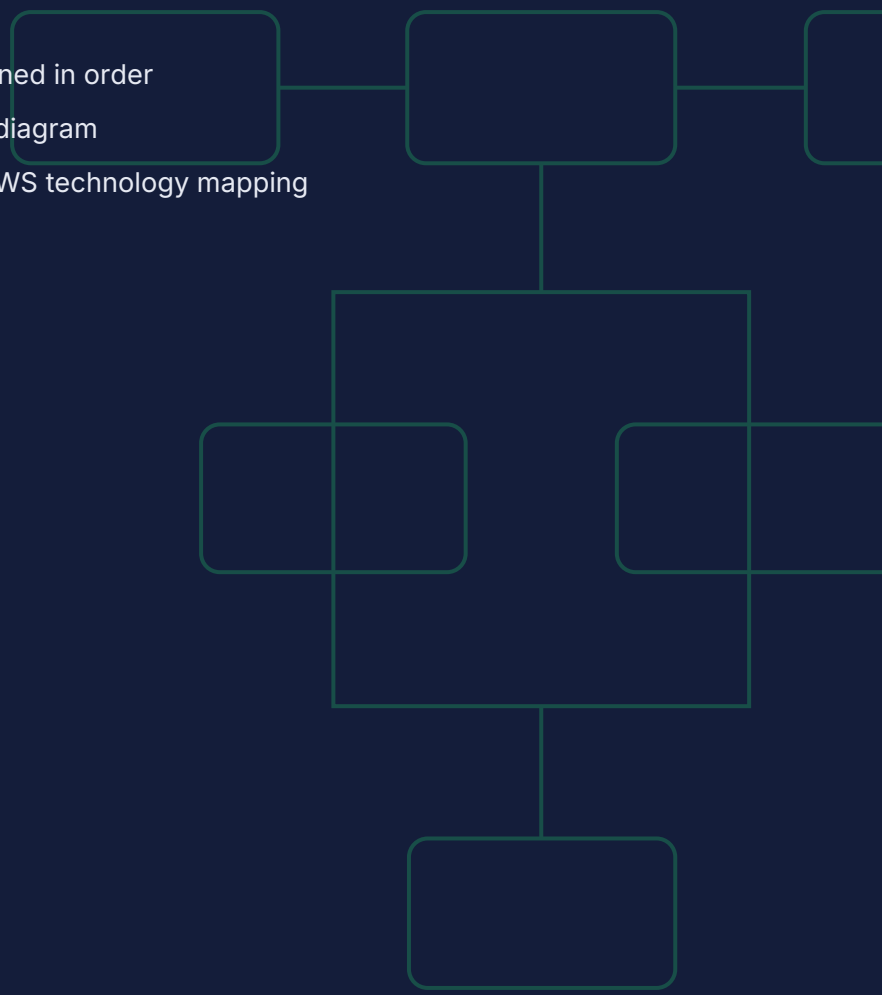
DATA ENGINEERING

CI/CD for Data Engineering

From code validation to real-time streaming and RAG: a production-ready pipeline that is tested, safely deployed, and continuously monitored.

- The 25 steps across four layers, explained in order
- The complete production architecture diagram
- CI/CD vs runtime workflows, with an AWS technology mapping
- A production-readiness checklist



START HERE

Why CI/CD for data engineering?

Modern data platforms do more than collect and store data. They process batch workloads, consume real-time events, validate data quality, power analytics, and support AI applications such as Retrieval-Augmented Generation (RAG).

But how do we make sure changes to these systems are tested, deployed safely, and monitored continuously without disrupting production? The answer is a well-designed **CI/CD pipeline combined with DataOps, automated data quality testing, runtime validation, and production observability**.

> **A successful code deployment does not mean the data pipeline is working correctly.**

Traditional CI/CD focuses on application code. Data engineering adds a second thing to protect: the data itself. A production-grade platform must validate both **the software and the data it processes**. CI/CD automates code validation and releases; DataOps extends the same discipline to data quality, pipeline reliability, and operational monitoring.

When a green deployment still goes wrong

Each of these can happen after every code check has passed:

Where it happens	What goes wrong
Spark job	Deploys successfully but produces incorrect results
Streaming application	Processes events out of order
API schema change	Breaks downstream consumers
RAG pipeline	Generates incorrect embeddings or reduces retrieval quality
Production pipeline	Runs successfully while processing stale or incomplete data

The four layers at a glance

Layer	When	What it does
A · Source control & CI validation	Before merge	Checks code, dependencies, schemas, data quality rules, streaming behavior, and RAG functionality before a release.
B · Build, package & release	Before production	Creates versioned artifacts, scans them, deploys to staging, runs integration tests, and promotes approved changes.
C · Runtime validation & freshness	While running	Processes real batch data, streaming events, and RAG documents while validating quality and freshness.
D · Observability & recovery	After deployment	Monitors health, evaluates data and retrieval quality, and triggers recovery or corrective changes.

These layers are connected. A release moves forward only when the relevant checks pass, while production feedback improves future releases.

STEP 1

The four layers, step by step

Twenty-five steps, grouped into four layers. Each card starts with a real situation, lists its steps in order, and ends with the questions to settle before you build it.

A Source control & CI validation

Steps 1–7 · Before merge

> **SITUATION** A pull request changes a Spark transformation. Before it can merge, the pipeline must prove that the code, schema, data rules, streaming behavior, and RAG logic are all safe, without relying on manual checks.

1	**Developer + Git repo**	Python, SQL, Spark or Flink jobs, connectors, consumers, and DAGs live in GitHub or GitLab, with feature branches, pull requests, and reviews.
2	**CI trigger**	A pull request, push, or approved merge starts predefined checks instead of manual validation.
3	**Static analysis + security**	Formatting, linting, type errors, SQL and config quality, dependency vulnerabilities, committed secrets, and infrastructure configuration.
4	**Unit + contract tests**	Unit tests cover functions, transformations, parsers, and business rules. Contract tests check APIs, schemas, and downstream interfaces.
5	**Data quality tests**	Schemas, nulls, duplicates, business rules, freshness, and PII handling. A missing transaction ID, for example, blocks publication.
6	**Streaming + RAG tests**	Event ordering, late events, replay, idempotency; chunking, embedding dimensions, retrieval relevance, and search regressions.
7	**CI gate**	Collects every required result. If checks pass the pipeline proceeds; if not, it blocks promotion and produces a test report.

KEY QUESTIONS

- Which checks must pass before a change can be promoted?
- Do CI tests use synthetic, masked, or approved fixtures rather than unrestricted production data?

B Build, package & release

Steps 8–14 · Before production

> **SITUATION** The change has passed CI. It must reach production as the exact same versioned, scanned, signed artifact that was tested, not a freshly rebuilt version that might differ.

8	**Build immutable artifacts**	Package code, jobs, DAGs, SQL migrations, and config as Docker images, Python jobs, or bundles, each with a unique version.
9	**Security + provenance**	Generate a Software Bill of Materials (SBOM), scan dependencies, sign artifacts, and record provenance.
10	**Artifact registry**	Store validated artifacts in a versioned registry such as Amazon ECR, and promote the same artifact through every environment.
11	**Deploy to staging**	A production-like environment, infrastructure-as-code plans, and secrets fetched from a secret manager.
12	**Integration + e2e tests**	Representative batch jobs, simulated streaming events, database integration tests, RAG smoke tests, and load tests.
13	**Approval + release gate**	Test results, security findings, operational readiness, and release policy decide; approval is human or automated depending on risk.
14	**Production deployment**	Rolling, blue-green, or canary, accounting for checkpoints, schema compatibility, duplicate processing, and safe job transitions.

KEY QUESTIONS

- Is the same approved artifact promoted from staging to production?
- Which releases need human approval, and which follow an automated policy?

C Runtime validation & freshness

Steps 15–20 · While running

> **SITUATION** The release is live, but a pipeline can run happily while processing stale or incomplete data. The CI/CD pipeline deploys the processing logic; the runtime pipeline processes the production data.

15	**Ingest from sources**	Batch (databases, exports, files, APIs), streaming (Kafka topics, event streams), and RAG documents (PDFs, text, knowledge bases).
16	**Ingestion + schema registry**	Check records against expected schemas and metadata; a schema registry lets producers and consumers agree on formats.
17	**Transform, validate, dedupe**	Parse, clean, and normalize; remove duplicates, reject or quarantine bad records. An event ID can catch duplicate transactions.
18	**Process + store**	Scheduled batch jobs and continuous streaming write to a lake or warehouse, planning for partitioning, replay, checkpoints, and late events.
19	**RAG ingestion pipeline**	Extract → Clean → Chunk → Embed → Store in a vector index, with permissions enforced and outdated chunks updated or removed.
20	**Post-deployment validation**	Record counts, freshness and lag, error rates, dead-letter volume, schema violations, and retrieval quality; alert, quarantine, or recover.

KEY QUESTIONS

- Which data must always be fresh, and what lag is acceptable?
- What happens to invalid records: rejected, or quarantined for review?

D Observability, rollback & improvement

Steps 21–25 · After deployment

> **SITUATION** A new streaming consumer causes a sharp rise in processing lag. The release must be halted while the team investigates, and the lesson must not be forgotten.

21	**Monitoring + alerts**	Infrastructure (CPU, memory, availability), data (lag, failed jobs, freshness, counts, cost), and RAG (relevance, latency, regressions).
22	**Evaluation**	Compare against baselines and a golden evaluation dataset, with thresholds set per workload rather than one for all pipelines.
23	**Release + recovery**	Roll back the image, revert config, restore a job version, or replay from a checkpoint. Rolling back code does not undo data already written.
24	**Feedback loop**	Incidents and failed checks become regression tests, updated validation rules, pipeline fixes, and a new pull request.
25	**Block release or roll back**	Stop promotion or trigger rollback when critical health thresholds are breached, with clear recovery policies.

KEY QUESTIONS

- What number triggers an alert, and what number triggers a rollback?
- Does the recovery plan cover schema migrations, duplicate events, and partial writes?

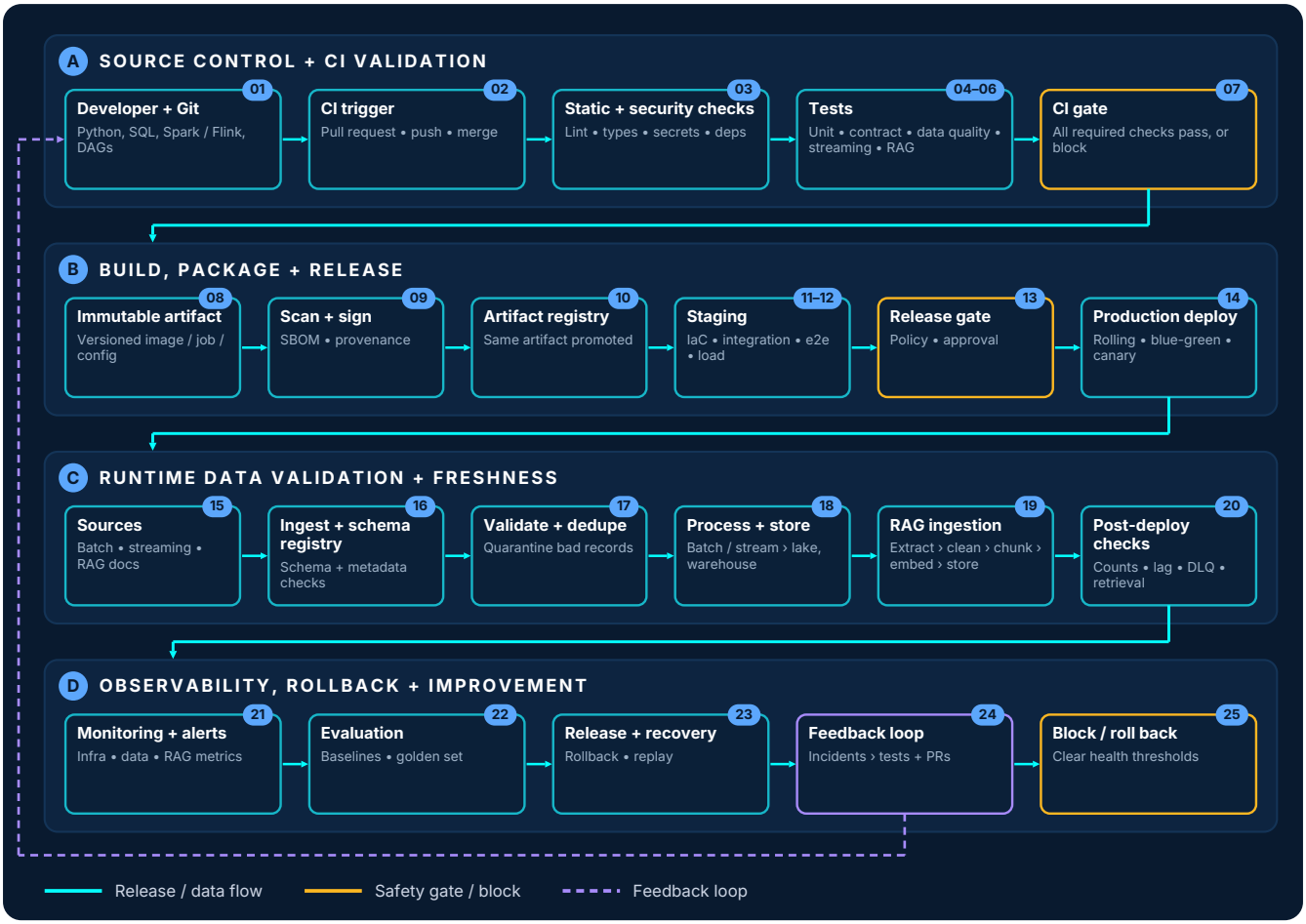
One change, end to end

Follow one pull request, a new streaming consumer, through the architecture:

1. **CI gate** runs every code, security, data quality, streaming, and RAG check, and blocks on any failure.
2. **Build** packages a versioned, scanned, signed artifact and stores it in the registry.
3. **Staging** runs integration, end-to-end, and load tests; the **release gate** weighs results, findings, and policy.
4. **Production deployment** uses a rolling, blue-green, or canary rollout that respects checkpoints.
5. **Runtime pipeline** ingests batch, streaming, and RAG data, validating schemas, duplicates, and freshness.
6. **Monitoring** tracks lag, errors, freshness, and cost, while **evaluation** compares them with baselines.
7. If lag spikes, the release is **halted or rolled back**, and the incident becomes a regression test.

STEP 2

The complete architecture



A · Source control & CI
Developer, Git, CI trigger, static and security checks, tests, and the CI gate.

B · Build & release
Immutable artifacts, SBOM and signing, registry, staging, approval, production deploy.

C · Runtime & freshness
Batch, streaming, and RAG ingestion with schema checks, dedupe, and post-deploy validation.

D · Observability & recovery
Monitoring, evaluation, rollback and replay, and the feedback loop into new tests.

Testing a pipeline vs running a pipeline

The most important idea in this architecture: the CI/CD pipeline deploys the processing logic, while the runtime data pipeline processes incoming production data. They are connected workflows, but they are not the same thing.

CI/CD workflow	Runtime data workflow
Validates code and configuration	Processes incoming production data
Uses unit tests and test fixtures	Uses real batch records and streaming events
Checks schema and contract compatibility	Validates incoming records against schemas
Tests RAG chunking and retrieval behavior	Ingests documents and updates embeddings
Builds and promotes release artifacts	Executes the deployed processing logic
Blocks unsafe releases	Alerts, quarantines, retries, or recovers from data failures

A CI pipeline can prove a transformation works against a test dataset. Runtime monitoring shows whether it keeps behaving correctly when real data arrives.

STEP 3

Implementation and checklist

Example AWS technology mapping

The architecture can be built on different stacks. These AWS services are possible choices, not mandatory requirements.

Architecture requirement	Example AWS technology
Source control	GitHub or AWS CodeCommit where available
CI/CD orchestration	AWS CodePipeline, CodeBuild, or Jenkins
Container registry	Amazon ECR
Batch and ETL processing	AWS Glue or Amazon EMR
Event streaming	Amazon MSK or Amazon Kinesis Data Streams
Data lake storage	Amazon S3
Data warehouse	Amazon Redshift
Workflow orchestration	Amazon MWAA or AWS Step Functions
Secrets management	AWS Secrets Manager
Monitoring and logs	Amazon CloudWatch
Infrastructure as code	AWS CloudFormation or Terraform
Vector retrieval	Amazon OpenSearch Service or another compatible vector database

The right selection depends on throughput, latency, existing infrastructure, operational expertise, and cost, and these components can be combined with open-source technologies.

Production-readiness checklist

Before you call a data platform production-ready, you should be able to tick every box.

- [ ] Automated quality gates stop critical code, security, and data validation failures before production
- [ ] Code, artifacts, schemas, pipeline configuration, and RAG ingestion logic are all versioned
- [ ] CI tests use synthetic, masked, or approved fixtures; the same approved artifact is promoted to production
- [ ] Least-privilege access, secure secret handling, masking, and retention controls are in place
- [ ] Replay and recovery are designed: duplicate events, checkpoints, partial writes, and failed batch jobs
- [ ] Freshness is monitored as well as availability, because a pipeline can run while its data is stale
- [ ] Canary or blue-green rollout, with health verified before full promotion
- [ ] Rollback plan covers data already written, including schema migrations and reconciliation or replay
- [ ] Outcomes are measured: deployment frequency, change failure rate, recovery time, data quality, latency, cost
- [ ] Production incidents are turned into regression tests and measurable improvements

> **Remember:** validate before deployment. Verify during execution. Monitor after deployment. Recover safely when something fails.

Based on AWS Prescriptive Guidance for data engineering and CI/CD: docs.aws.amazon.com/prescriptive-guidance (aws-caf-platform-perspective: data-eng, ci-cd).